



BITS Pilani
DIGITAL

M.Sc. in Data Science and AI

Cloud Services & Platform

BUILDING A SERVERLESS DATA PROCESSING PIPELINE ON AWS

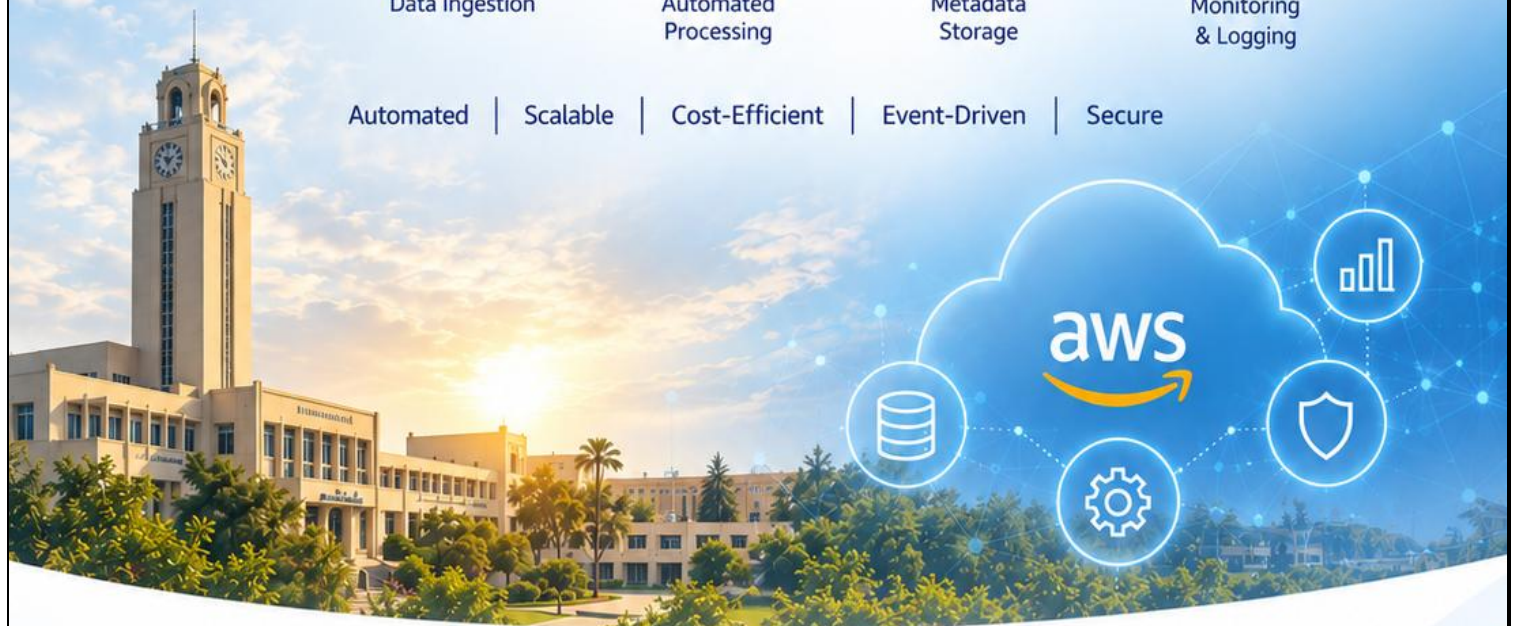
LEARN
BUILD
INNOVATE

Project Report

CLOUD
FOR A
BETTER
TOMORROW



Automated | Scalable | Cost-Efficient | Event-Driven | Secure



Submitted by: Sunita Baloda

Table of Contents

1. Executive Summary 3

2. Project Objective and Scope 3

3. High-Level Solution Architecture 3

4. End-to-End Data Processing Workflow 4

5. Lambda Processing Lifecycle 5

6. AWS Services and Their Responsibilities 8

7. Security and IAM..... 9

8. Observability and End-to-End Validation.....11

9. Cost and Scalability Considerations12

10. Cloud Architecture Principles Demonstrated13

11. Implementation Evidence14

12. Key Learning Outcomes14

13. Possible Future Enhancements15

14. Conclusion.....15

Appendix A — Visualization Index.....15

1. Executive Summary

This project demonstrates the design and implementation of a serverless, event-driven data processing pipeline on Amazon Web Services (AWS). The solution uses Amazon S3 as the data ingestion layer, AWS Lambda for automated file processing, Amazon DynamoDB for metadata storage, and Amazon CloudWatch for logging and monitoring. AWS Identity and Access Management (IAM) is used to control access to the resources involved in the workflow.

A CSV upload to the S3 bucket acts as the triggering event. The Lambda function reads the uploaded file, validates that it is a CSV, calculates basic file metrics such as row and column counts, generates metadata, and stores the resulting metadata in DynamoDB. CloudWatch provides visibility into Lambda execution through logs.

The project provides a practical demonstration of serverless computing, event-driven architecture, managed cloud services, loose coupling, automated processing, metadata storage, observability, access control, scalability, and cost-aware cloud design.

2. Project Objective and Scope

The objective of the project is to build and validate a serverless data-processing workflow that automatically responds to a file upload and processes the uploaded data using AWS managed services.

- Use Amazon S3 as the file-ingestion and storage layer.
- Trigger AWS Lambda automatically when a new CSV file is uploaded.
- Read and validate the uploaded CSV using Python/boto3.
- Calculate basic file information such as file name, file type, file size, rows, columns and timestamp.
- Store the resulting metadata in Amazon DynamoDB.
- Use Amazon CloudWatch for execution logging and monitoring.
- Apply IAM-based access control to the resources required by the workflow.

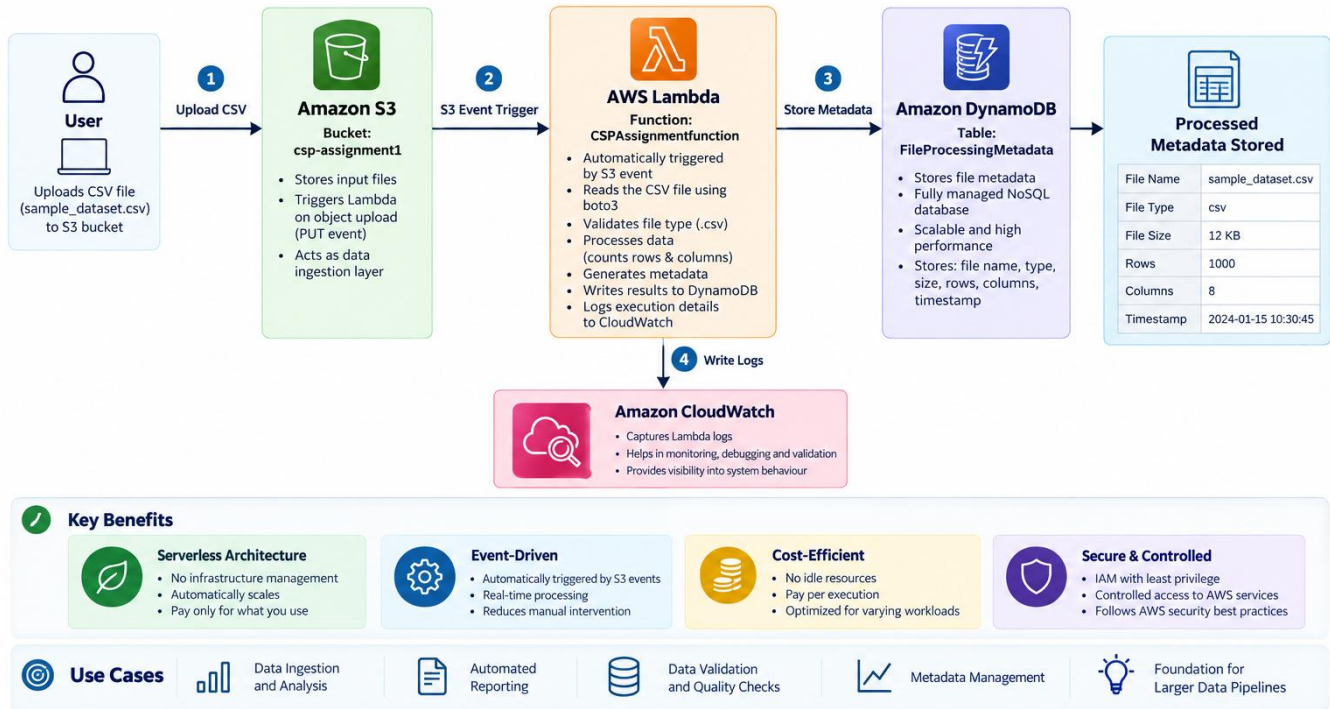
3. High-Level Solution Architecture

The solution uses an event-driven serverless architecture. The user provides the input through an S3 upload; the S3 event invokes Lambda; Lambda performs the file-processing work; DynamoDB stores the generated metadata; and CloudWatch provides execution visibility.

AWS Serverless Data Processing Pipeline

Automated • Scalable • Cost-Efficient • Event-Driven

Upload a CSV file to Amazon S3 → Trigger AWS Lambda → Process data → Store metadata in DynamoDB → Monitor with CloudWatch



A simple file upload can power a scalable, intelligent, and fully automated data pipeline on AWS.

Figure 1. High-Level AWS Serverless Architecture

The architecture separates ingestion, processing, metadata storage, monitoring, and access management into distinct responsibilities. This reduces the need for a continuously running traditional server and demonstrates loose coupling between components.

4. End-to-End Data Processing Workflow

The workflow begins with a CSV file upload and proceeds automatically through the processing pipeline.

1. A CSV file is uploaded to the Amazon S3 bucket.
2. The S3 PUT event invokes the Lambda function.
3. Lambda reads the uploaded object from S3 using boto3.
4. The function validates that the input is a CSV file.
5. The function processes the file and calculates basic metrics.
6. Metadata is generated from the processed file.
7. The metadata is stored in Amazon DynamoDB.
8. Lambda execution information is captured in Amazon CloudWatch.

End-to-End Workflow

Serverless Data Processing Pipeline on AWS

From file upload to insights – fully automated and event-driven

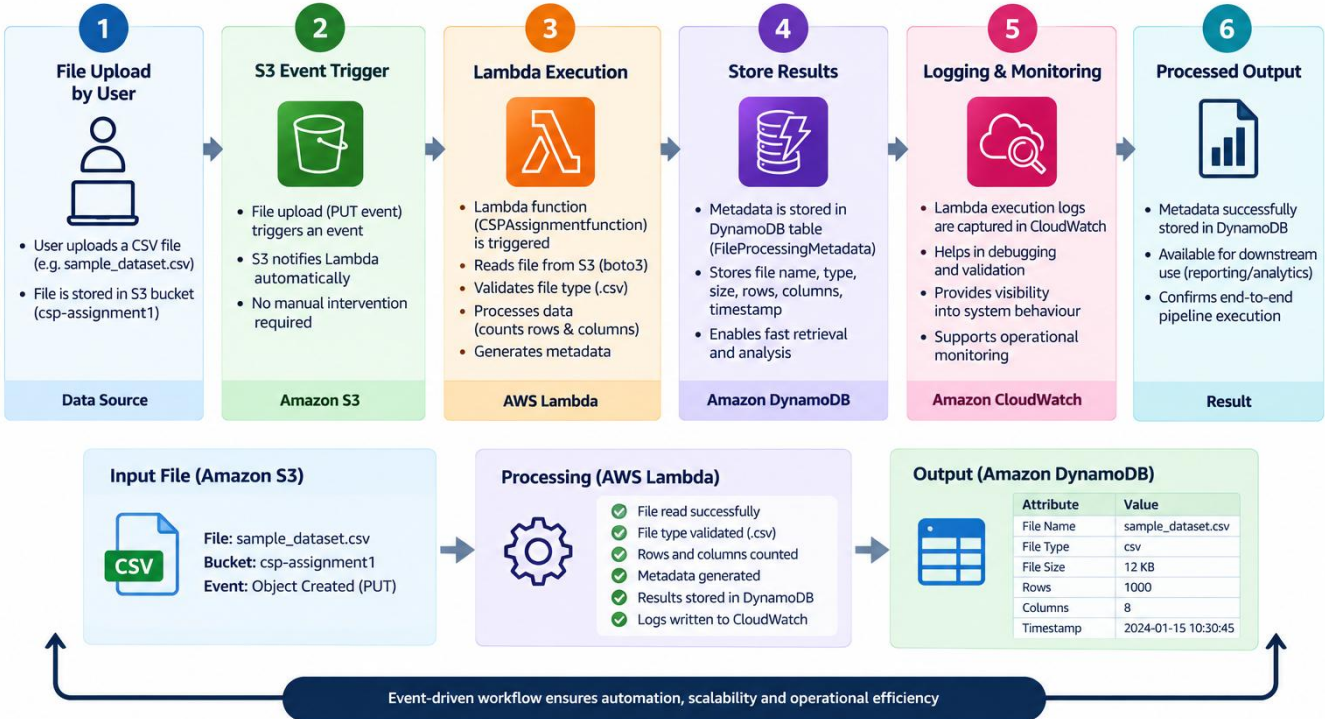
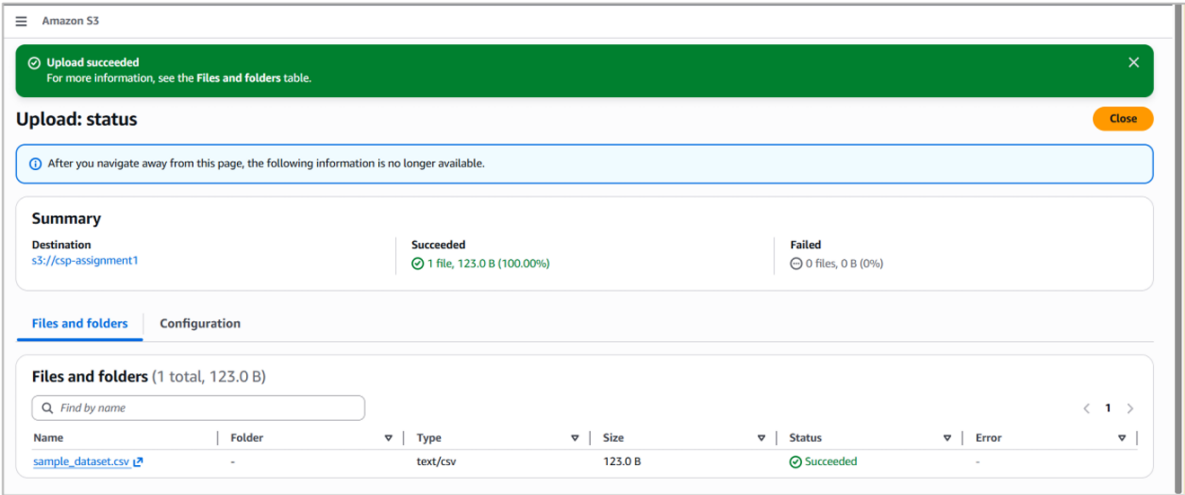


Figure 2. End-to-End Processing Workflow



CSV file uploaded successfully to the S3 bucket

5. Lambda Processing Lifecycle

AWS Lambda is the processing component of the architecture. The function documented in the project is named CSPAssignmentfunction and is invoked automatically following the S3 upload event.

AWS Lambda Processing Lifecycle

Inside the Lambda Function – From Trigger to Results

This diagram shows the step-by-step execution flow of the AWS Lambda function that processes the CSV file uploaded to Amazon S3.

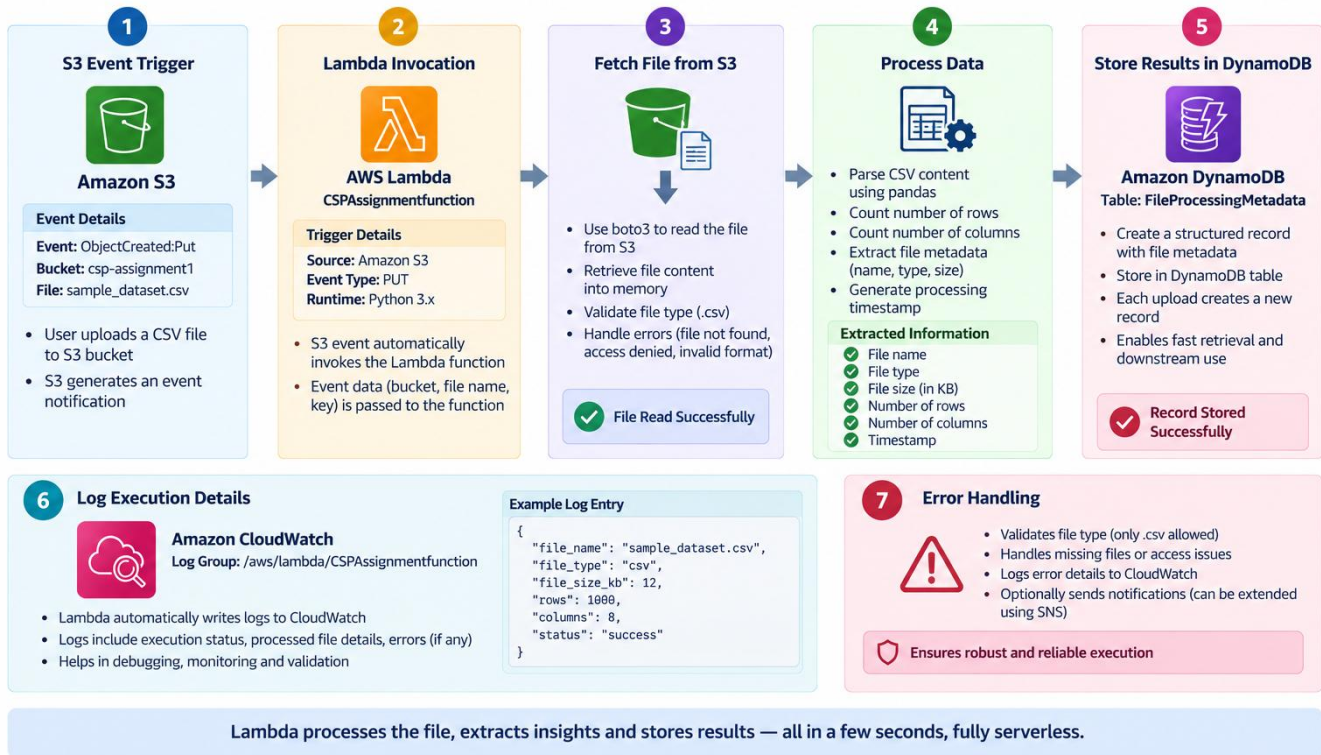
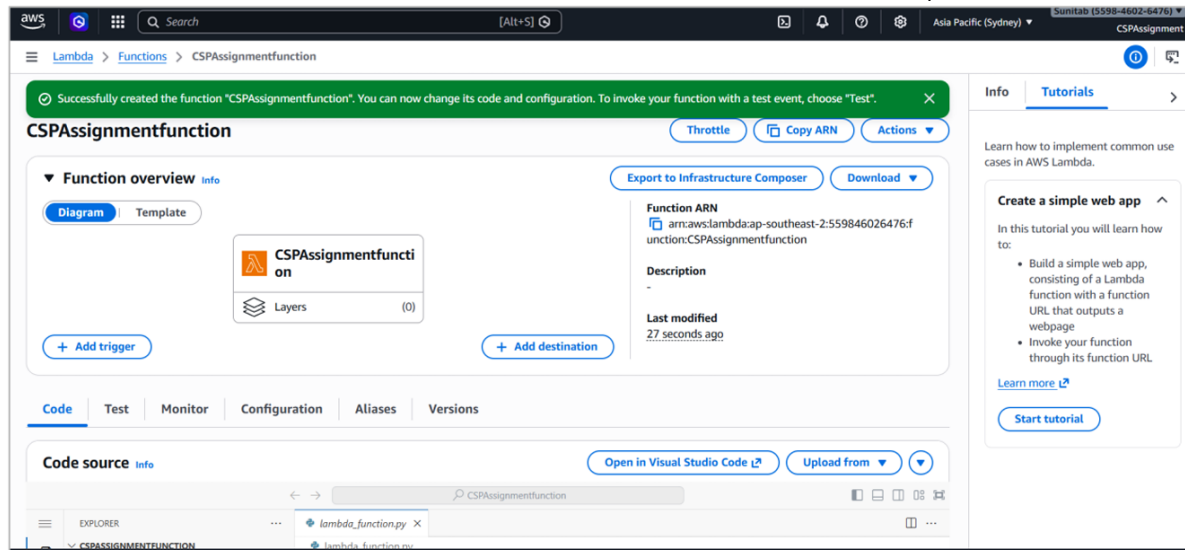


Figure 3. Lambda Processing Lifecycle

5.1 Processing Responsibilities

- Receive the S3 event notification.
- Identify the uploaded object.
- Read the file from S3 using boto3.
- Validate the input file type.
- Calculate the number of rows and columns.
- Generate structured metadata including file name, type, size and timestamp.
- Write the resulting metadata to DynamoDB.
- Record execution information in CloudWatch.



AWS Lambda function CSPAssignmentfunction and its configuration

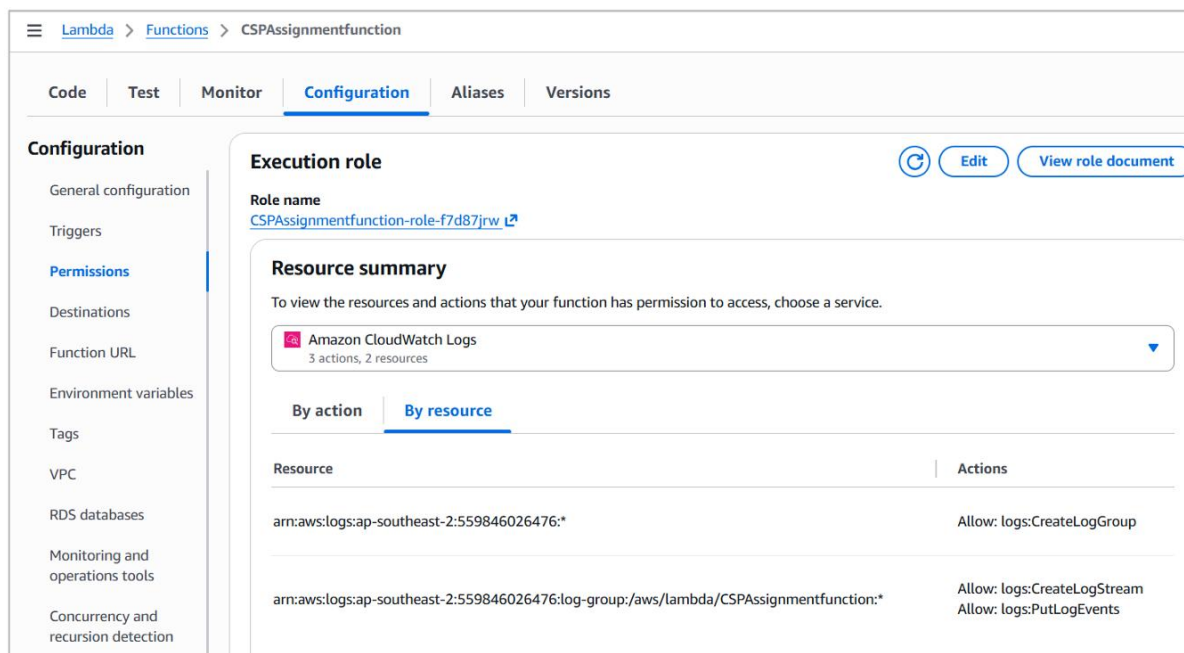
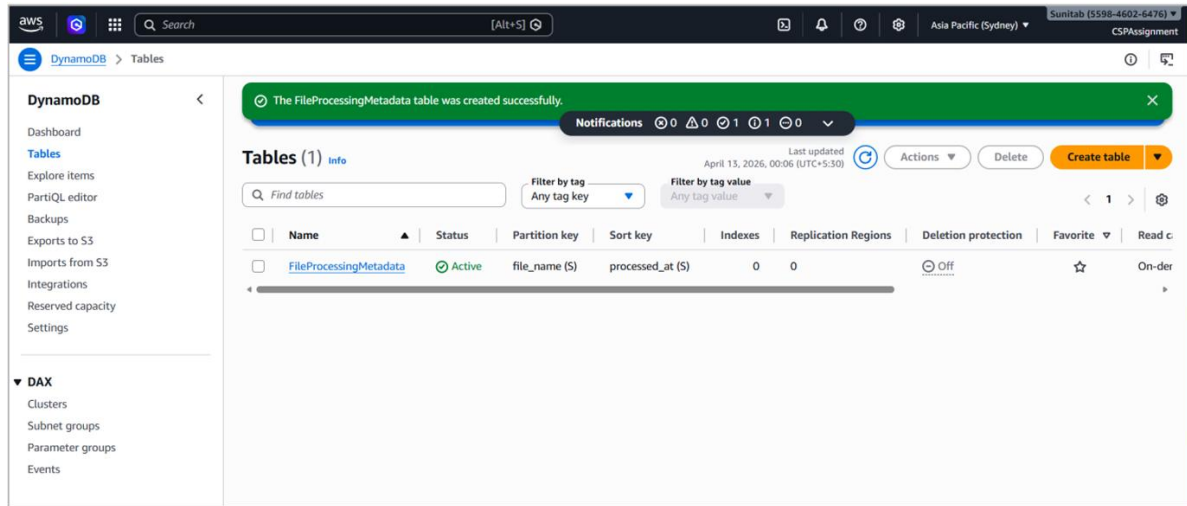


Figure 8. Lambda execution-role permissions supporting the serverless workflow



DynamoDB table created for FileProcessingMetadata

6. AWS Services and Their Responsibilities

Service	Project Component	Responsibility
Amazon S3	csp-assignment1	Receives and stores uploaded CSV files; acts as the ingestion layer.
AWS Lambda	CSPAssignmentfunction	Automatically processes files after an S3 event.
Amazon DynamoDB	FileProcessingMetadata	Stores structured metadata generated during processing.
Amazon CloudWatch	Lambda logs	Provides execution logs for monitoring and debugging.
AWS IAM	User / execution permissions	Controls identities, roles and permissions required by the workflow.

AWS Service Responsibility Map

Key Services and Their Roles in the Serverless Data Processing Pipeline

This diagram shows the core AWS services used in the project and the specific responsibilities of each service in enabling a secure, scalable and fully serverless data processing pipeline.

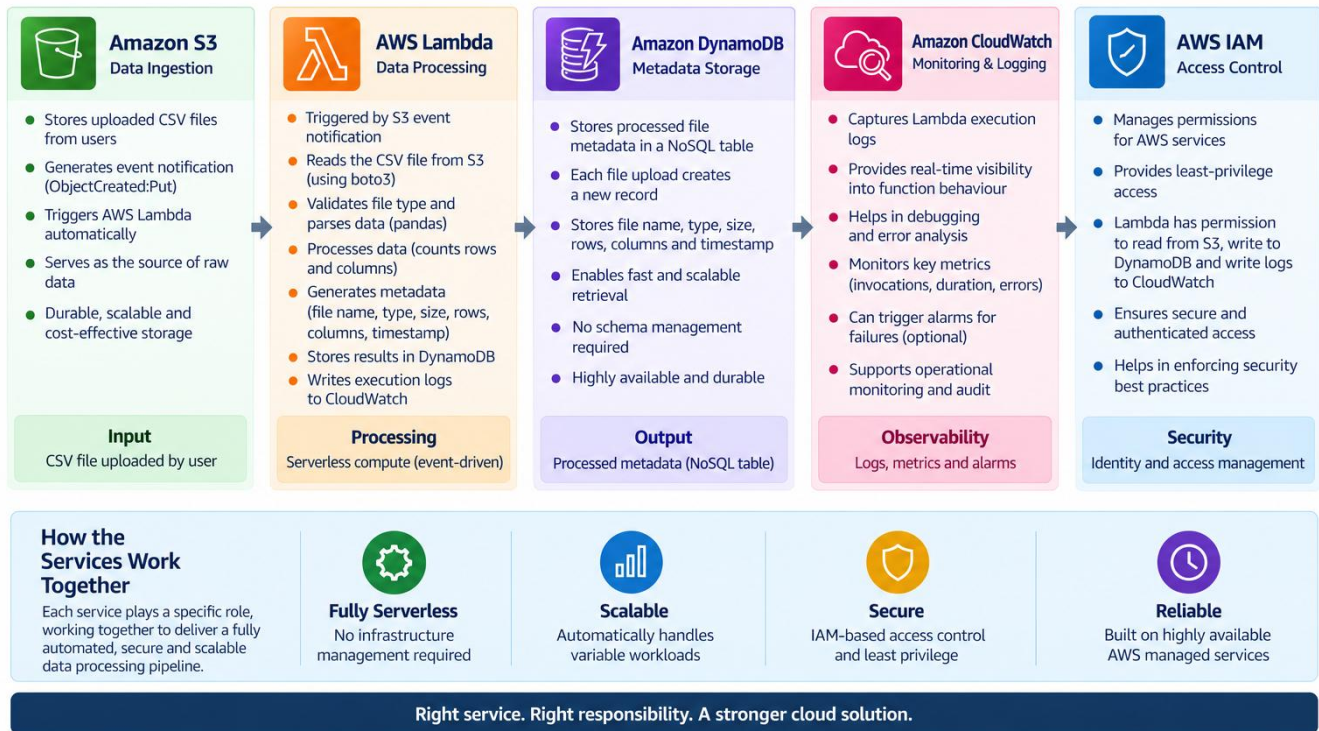


Figure 4. AWS Service Responsibility Map

7. Security and IAM

IAM is used to control access to the AWS resources required by the implementation. The project uses a dedicated IAM-based access model and a Lambda execution role for service interaction.

- Controlled access to AWS resources through IAM.
- A Lambda execution role supplies permissions required by the function.
- Permissions are scoped to the resources needed by the workflow.
- The design follows the least-privilege principle as a security objective.
- Application processing and access management are kept as separate responsibilities.

Security and IAM Model

Secure, Controlled and Least-Privilege Access to AWS Resources

This diagram shows the IAM setup and access control model used in the serverless data processing pipeline, ensuring secure interaction with AWS services while following the principle of least privilege.

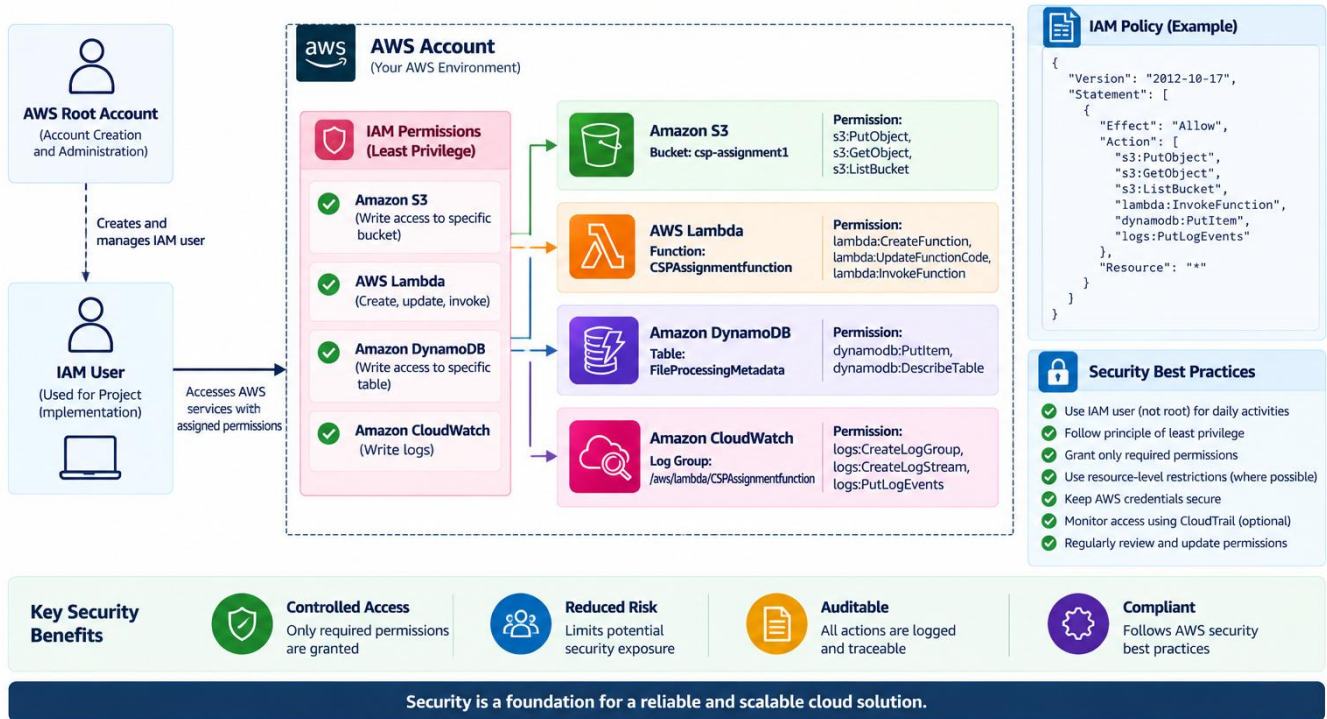
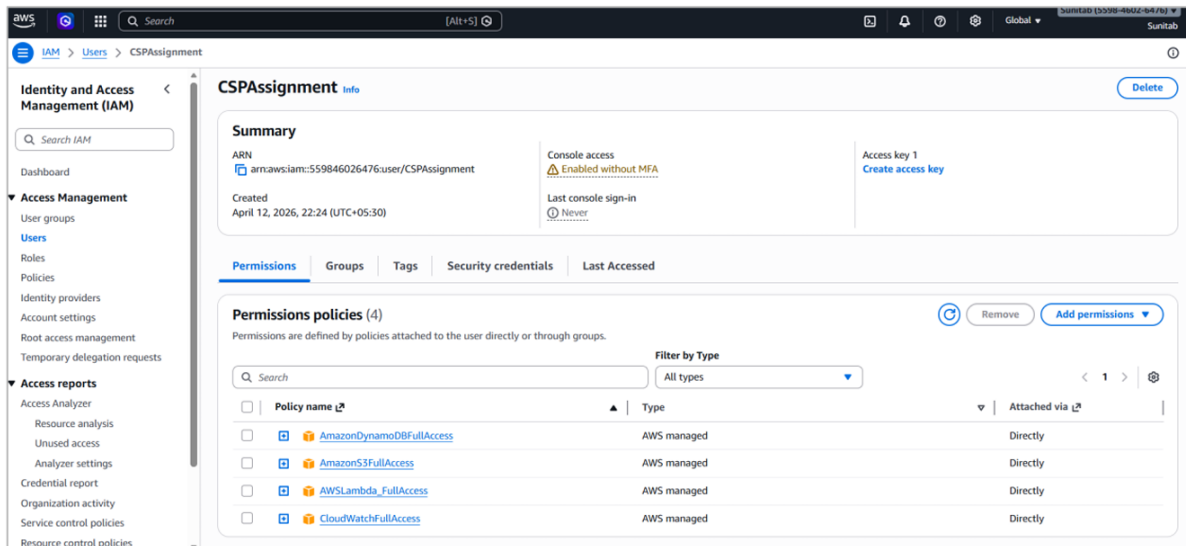


Figure 5. Security and IAM Model



IAM permissions configured for the project user/execution workflow.

8. Observability and End-to-End Validation

Amazon CloudWatch provides visibility into Lambda execution and supports troubleshooting and validation. The project documentation records an end-to-end validation in which the upload, Lambda invocation, CloudWatch logs and DynamoDB update were observed.

8.1 Validation Sequence

- Upload the CSV file to S3.
- Confirm the S3 event invokes CSPAssignmentfunction.
- Verify that Lambda executes and processes the file.
- Check CloudWatch for the corresponding execution logs.
- Verify that the generated metadata is written to FileProcessingMetadata.

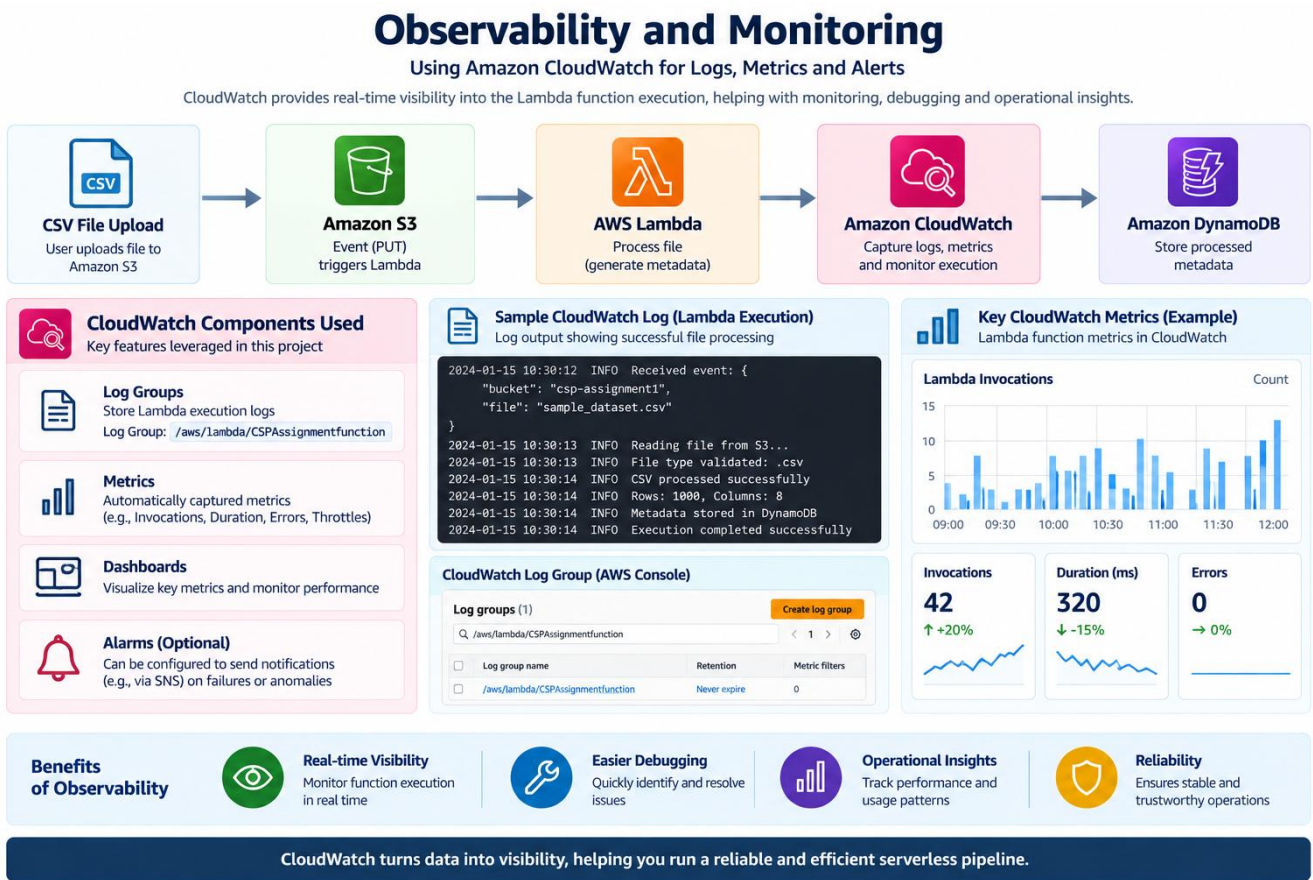
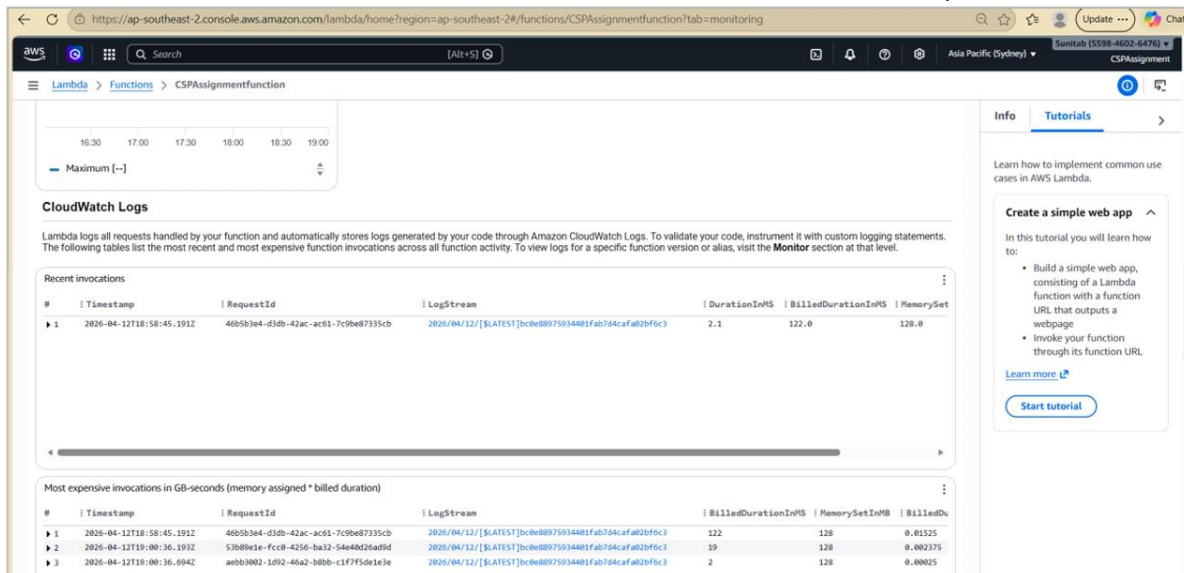
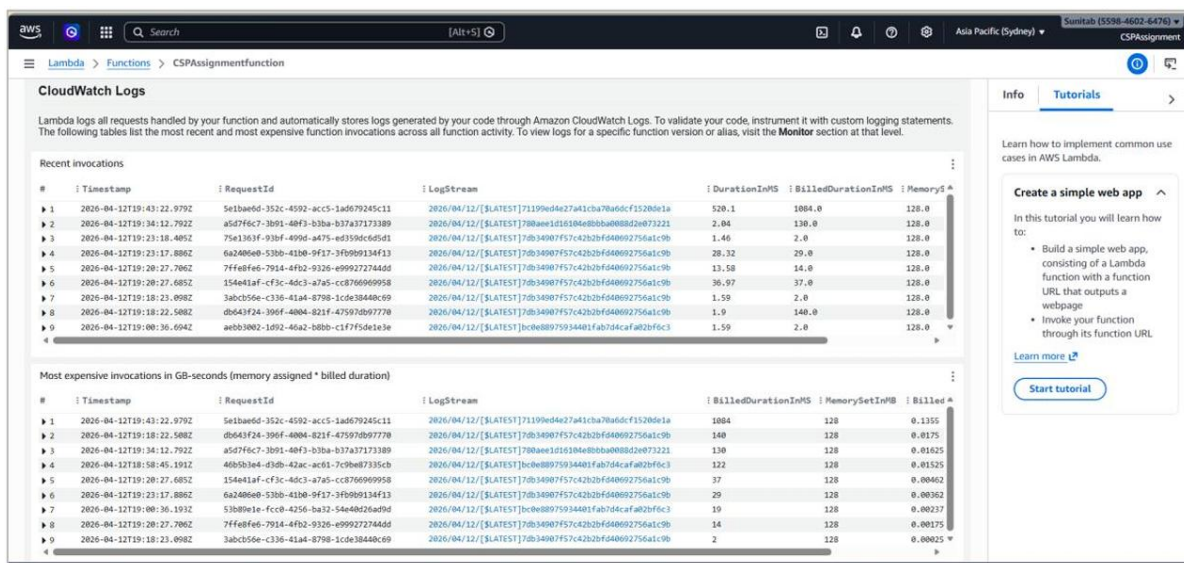


Figure 6. Observability and Validation Flow



CloudWatch log view for the Lambda execution



. Logged execution information demonstrating observability

9. Cost and Scalability Considerations

The serverless architecture avoids the need to provision and continuously operate traditional application servers. The design uses event-driven execution and managed AWS services, making it appropriate for workloads where demand can vary over time.

- On-demand serverless execution through AWS Lambda.
- No traditional application server needs to be continuously provisioned for the processing workflow.
- Managed services reduce infrastructure-management overhead.
- The event-driven design can accommodate additional file-processing events.
- Usage-based cloud services support cost-aware architecture for intermittent workloads.

Cost Estimation and Scalability

A Cost-Effective and Auto-Scaling Serverless Solution

This diagram shows the estimated cost for running the serverless data processing pipeline and how the solution automatically scales to handle varying workloads.



Figure 7. Cost and Scalability Characteristics

The cost visualisation is an architecture-level representation, not a billing statement for the deployment. Actual AWS charges depend on usage, AWS Region, account status and applicable pricing/free-tier conditions.

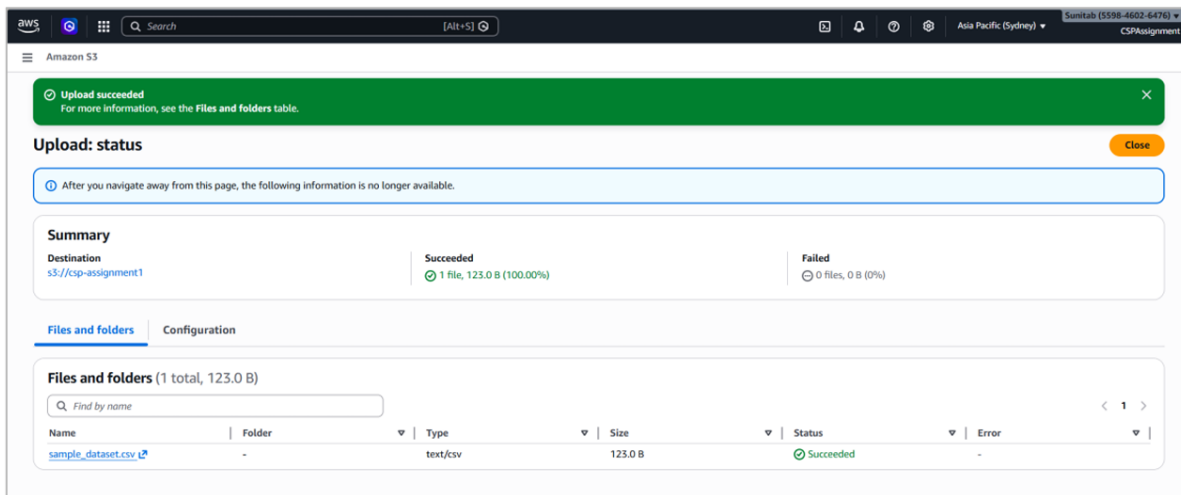
10. Cloud Architecture Principles Demonstrated

Principle	How the Project Demonstrates It
Event-driven architecture	An S3 upload event automatically initiates processing.
Serverless computing	Lambda executes processing without requiring a continuously running server.
Loose coupling	S3, Lambda, DynamoDB and CloudWatch have separate responsibilities.
Managed services	Storage, compute, database and monitoring are provided through AWS managed services.
Observability	CloudWatch logs provide visibility into Lambda execution.
Security	IAM provides controlled access and execution permissions.
Scalability	Event-driven managed services can handle changing processing demand without manual server provisioning.
Cost awareness	Serverless execution avoids continuously running traditional compute infrastructure.

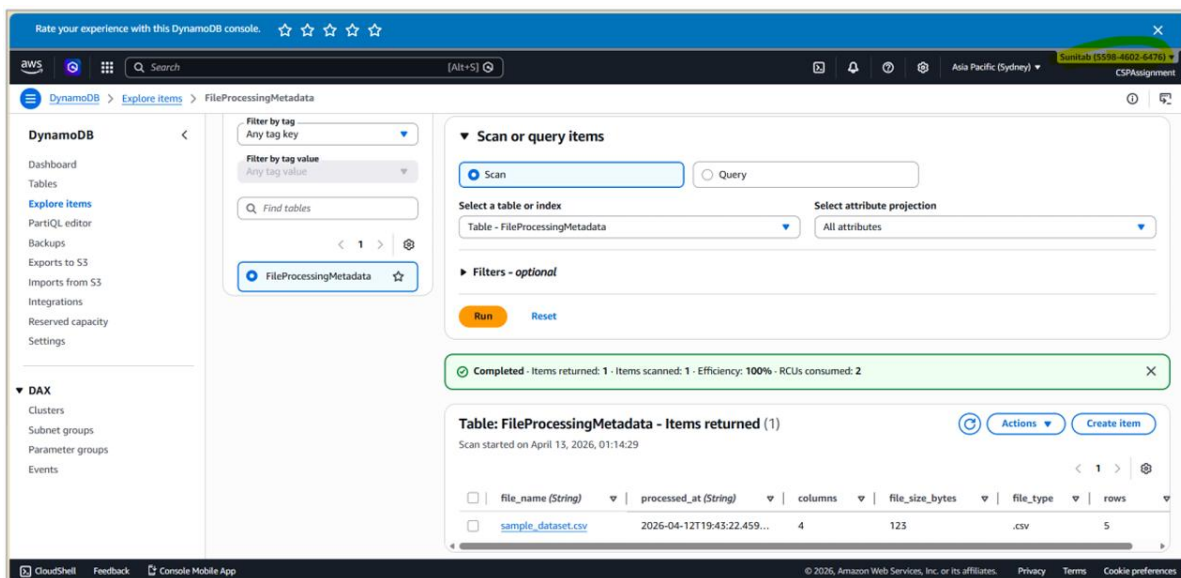
11. Implementation Evidence

The original project submission contains AWS console evidence covering the main implementation stages. The evidence includes IAM setup, S3 configuration and upload, Lambda configuration and permissions, DynamoDB creation, CloudWatch logging and end-to-end validation.

Component	Documented Value
S3 Bucket	csp-assignment1
Lambda Function	CSPAssignmentfunction
DynamoDB Table	FileProcessingMetadata
Sample Input	sample_dataset.csv



S3 upload status used during end-to-end validation.



Additional AWS console evidence from the implementation

12. Key Learning Outcomes

- Practical integration of multiple AWS managed services into one automated workflow.
- Understanding of event-driven execution using Amazon S3 and AWS Lambda.
- Experience using boto3 for interaction with AWS resources.
- Practical use of Amazon DynamoDB for structured metadata storage.
- Use of CloudWatch for observability, validation and debugging.
- Understanding of IAM roles and permission-based access.
- Understanding of serverless architecture, loose coupling and scalable cloud design.
- Awareness of cost considerations associated with serverless cloud solutions.

13. Possible Future Enhancements

The implemented solution is focused on the assignment workflow. The architecture could be extended without changing its core event-driven design.

- Add more extensive data-quality validation before metadata storage.
- Support additional input formats where required.
- Introduce downstream analytics or reporting after successful processing.
- Add alerting for failed Lambda executions.
- Extend the metadata model with additional file-quality and processing statistics.
- Integrate additional AWS services for larger production-oriented workflows.

14. Conclusion

The project demonstrates a serverless, event-driven data processing pipeline using Amazon S3, AWS Lambda, Amazon DynamoDB and Amazon CloudWatch, supported by IAM-based access control.

A CSV upload initiates automated processing, Lambda performs the file-processing logic, DynamoDB stores the resulting metadata, and CloudWatch provides execution visibility. The architecture demonstrates how managed cloud services can be combined to build an automated workflow without requiring a continuously running traditional server.

Overall, the project provides practical evidence of serverless computing, event-driven processing, managed storage, monitoring, security, scalability and cost-aware cloud architecture.

Appendix A — Visualization Index

- Figure 1. High-Level AWS Serverless Architecture — 01_high_level_aws_architecture_updated.png
- Figure 2. End-to-End Processing Workflow — 02_end_to_end_workflow.png
- Figure 3. Lambda Processing Lifecycle — 03_lambda_processing_lifecycle.png
- Figure 4. AWS Service Responsibility Map — 06_aws_service_responsibility_map.png
- Figure 5. Security and IAM Model — 04_security_iam_model.png
- Figure 6. Observability and Validation Flow — 05_observability_validation_flow.png
- Figure 7. Cost and Scalability Characteristics — 07_cost_estimation_and_scalability_updated.png